Tony Yiding Tian

Philadelphia, PA | (267)249-1202 | tonytg@engineering.upenn.edu | tonyxtian.com

Education

University of Pennsylvania Philadelphia, PA

B.S.E. in Computer Engineering & Accelerated M.S.E. in Computer Graphics *May 2027*

- GPA: 3.79 — Relevant Courses: GPU Programming, 3D Modeling, Advanced Rendering, Computer Animation, UI/UX Design, Procedural Graphics, Operating Systems Implementation, Data Structures & Algorithms

Experience

UI Rendering GPU Software Engineer Intern May 2026 – Aug 2026

Apple *Cupertino, CA*

- Designed and shipped internal developer tooling for Apple's UI rendering framework of Core Animation, taking it from proposal through design, implementation, on-device validation, and multi-round code review with senior engineers toward integration into a shared release branch
- Built cross-platform CLI tooling and wrote internal architecture documentation of the subsystems involved, verified against source, to make the workflow usable by other engineers after the internship. Tool accelerates certain internal development workflow speed by 4x and is merged into code base main branch
- Screened, triaged, and fixed various system UI rendering bugs reported throughout the iOS/macOS 27.0 beta releases. Actively collaborated with other Apple internal teams to minimize regression before each public release

Projects

CUDA Path Tracer - 3D PBR Renderer | *CUDA, GLSL, C++* Sep 2025 - Oct 2025

- Monte Carlo path tracer capable of rendering complex 3D scenes with custom 3D models and environment maps
- Implemented shading BSDF kernel supporting global illumination, multiple importance sampling, anti-aliasing, sub-surface scattering, capable of rendering various PBR material types with albedo and texture maps
- Integrated third-party libraries of tinyGLTF to support glTF 2.0 mesh loading and Nvidia OptiX for denoising
- Utilized various techniques to boost performance: material sorting (+5%), Russian Roulette (+6% - 24%), stream compaction (+24% - 67%), and Bounding Volume Hierachy (3× - 160× framerate in complex scenes)
- Project Repo and Demo: github.com/tonytgrt/CUDA-Path-Tracer. A previous standalone performance focused stream compaction project with detailed analysis in Nsight: github.com/tonytgrt/Project2-Stream-Compaction

Vulkan Grass Renderer - Physically Based Grass Simulation | *Vulkan, GLSL, C++* Oct 2025 – Nov 2025

- Implemented real-time grass rendering system in Vulkan, simulating up to 1 million grass blades with physics-based animation at interactive frame rates using compute and tessellation pipelines
- Developed GPU compute shader for physics simulation applying gravity, Hooke's law recovery forces, and wind with spatial turbulence to quadratic Bezier curve grass blades, processing 1M+ blade state updates per frame
- Engineered three-tiered GPU culling system (orientation, view-frustum, distance-based probabilistic) achieving 4.31× performance improvement at 1M blades through efficient atomic operations and indirect draw calls
- Built hardware tessellation pipeline with dynamic LOD based on camera distance, generating smooth blade geometry with exponential falloff from 10 to 2 subdivisions for optimal geometry density
- GitHub Repo: github.com/tonytgrt/Vulkan-Grass-Renderer

WebGPU Renderer - Real-time Renderer on Web | *WebGPU, TypeScript, WGSL* Sep 2025 – Oct 2025

- Implemented three advanced rendering techniques for real-time lighting of 5000+ dynamic point lights: Naive Forward, Forward+, and Clustered Deferred rendering using WebGPU compute and graphics pipelines
- Engineered screen-space light clustering system using compute shaders, subdividing view frustum into 16×9×24 grid with exponential depth slicing and sphere-AABB intersection testing for efficient light culling
- Developed G-buffer architecture with 3 render targets (position, normal, albedo) enabling two-pass deferred rendering that decouples geometry complexity from lighting calculations
- Achieved 53x performance improvement over naive rendering (497ms to 9.3ms at 5000 lights) and 3.5x speedup vs Forward+ through overdraw elimination and optimized memory access patterns
- Built automated performance testing system collecting statistical frame time data across 30 configurations (3 renderers × 10 light counts), generating CSV analysis for rigorous benchmarking
- Live demo deployed at webgpu.tonyxtian.com, rendering Sponza scene with real-time performance metrics and interactive controls. GitHub repo: github.com/tonytgrt/Project4-WebGPU-Forward-Plus-and-Clustered-Deferred

Technical Skills

Programming: C++, Objective-C, CUDA, MSL, GLSL, WGSL, SwiftUI, TypeScript, Java, Python, Bash
Graphics/Rendering: Core Animation, UI Compositing, GPU Profiling, Path Tracing, Deferred Rendering, PBR
Tools/APIs: Claude Code, XCode, Metal, Unreal, Unity, Maya, Nvidia NSight, WebGPU, Vulkan, OpenGL